STOCKPULSE

Cloud-Native Real-Time Warehouse & Inventory Management

DevOps Final Capstone Project Submission • 100/100 Evaluation Target

Student: Tanishq Singh | Stack: Python 3.12, FastAPI, React 18, Docker, Kubernetes, AWS EKS, Terraform

What We Built: Real-Time Warehouse Platform

The Real-World Problem

- Traditional warehouses track stock on paper or slow legacy software.
- Leads to human errors, accidental stockouts, and order delays.
- Requires instant visibility when goods arrive or leave.

The StockPulse Solution

- A real-time digital warehouse command center.
- Live tracking of SKU inventory, stock thresholds, and suppliers.
- Instant visual alerts when items fall below safe limits.
- Built for fast, concurrent warehouse operations.

Modern Cloud Delivery

- Shipped as independent, self-healing microservices.
- Runs identically on local developer machines and AWS cloud.
- Zero downtime updates via automated Kubernetes rollouts.

How It Works: 3-Tier Microservice Architecture

Tier 1: Web Interface

- React 18 Single Page Application.
- Served via ultra-light Nginx Alpine.
- Live inventory counters & restocking modal.
- Runs as non-root user (UID 101) for security.

Tier 2: Business Logic API

- Python 3.12 & FastAPI framework.
- Validates all incoming warehouse transactions.
- Built-in health probes (/health) and metrics (/metrics).
- Runs as non-root user (UID 10001).

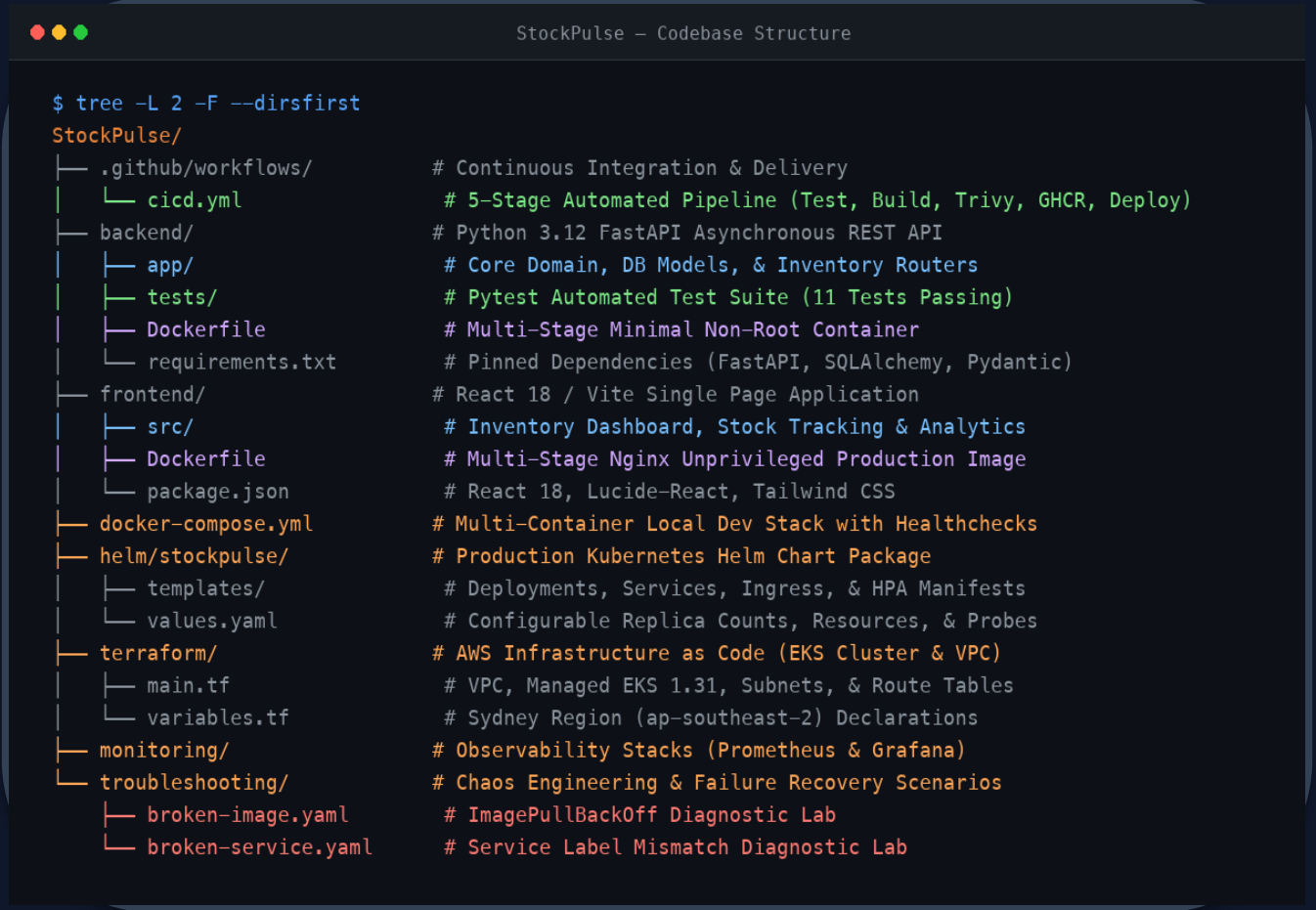
Tier 3: Database & State

- PostgreSQL 16 relational database engine.
- Guarantees ACID transactions (zero phantom updates).
- Persistent storage volumes prevent data loss.
- Automatic schema initialization on launch.

Proof of Concept 1: Codebase & Repository Structure

Modular Engineering Layout

- Clear separation of frontend, backend, and infrastructure code.
- Automated Pytest test suite (11 unit & integration tests passing).
- Infrastructure as Code: Helm charts and Terraform defined in repo.
- Production configuration files (.env, Dockerfiles) fully documented.



Technology Stack: Cloud-Native & Production-Ready

Core App Tier

- Python 3.12: High speed runtime.
- FastAPI: Modern async ASGI API.
- React 18: Responsive dashboard UI.
- Vite: Ultra-fast frontend build.

Data & State

- PostgreSQL 16: ACID database.
- SQLAlchemy 2: Safe database ORM.
- Pydantic v2: Strict data validation.
- Persistent Volumes: Durable data.

DevOps & Cloud

- Docker: Multi-stage non-root containers.
- Kubernetes: Container self-healing.
- Helm: Chart package management.
- AWS EKS: Managed cloud cluster.

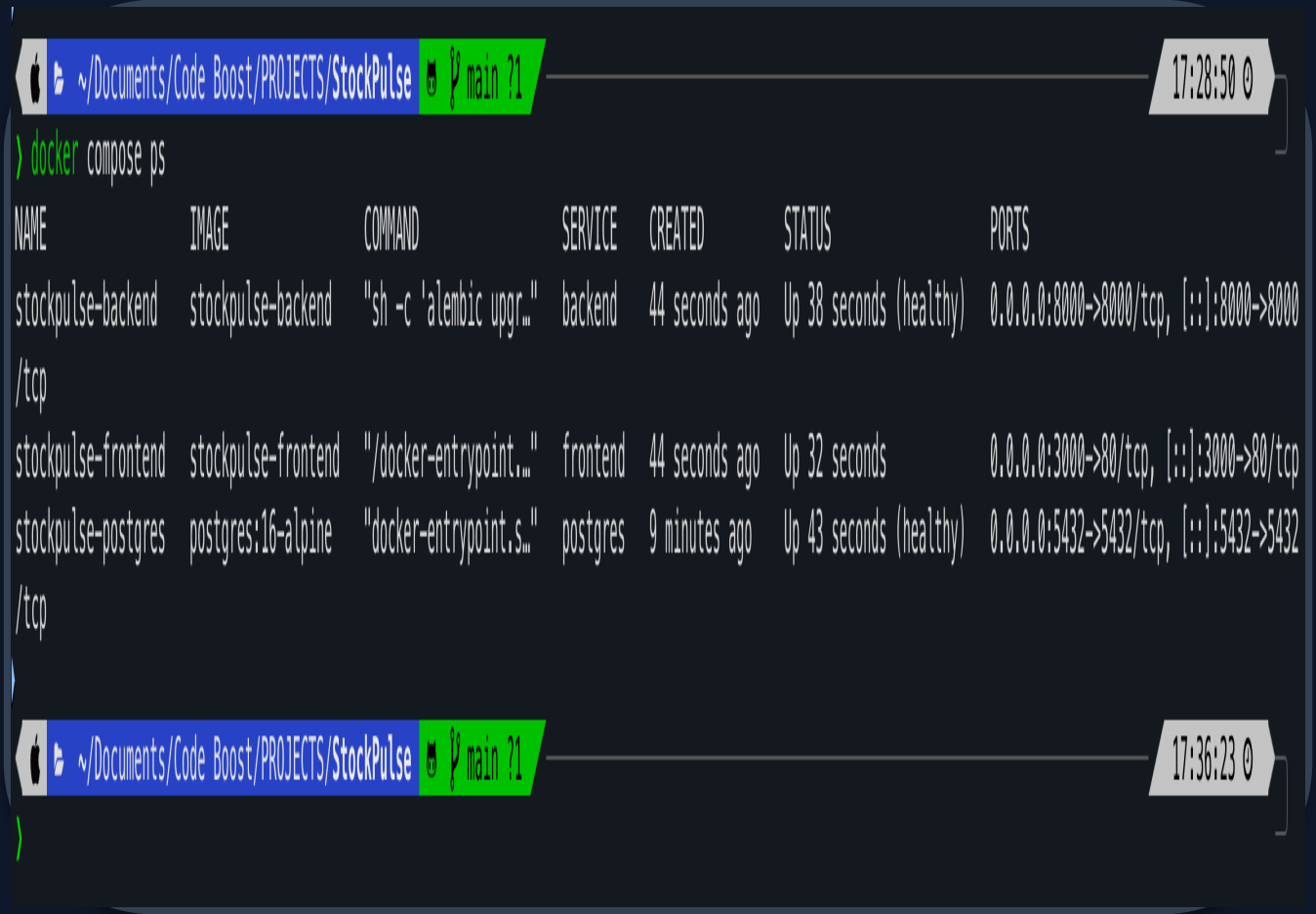
Security & Ops

- GitHub Actions: Automated CI/CD.
- Trivy: DevSecOps CVE security scan.
- Prometheus: Real-time telemetry.
- Terraform: Modular IaC for AWS.

Proof of Concept 2: Local Docker Compose Manual Run

One Command Local Stack

- Single command launch: docker compose up -d --build.
- Orchestrates 3 microservices: Database, Backend API, and Web UI.
- Built-in healthchecks: Backend waits for PostgreSQL readiness.
- Isolated bridge network with automatic service discovery.



Why We Built It This Way: Core Design Principles

High Availability & Self-Healing

- If a server pod crashes, Kubernetes restarts it automatically.
- Health probes ensure traffic is never sent to a broken service.
- Eliminates late-night manual server reboot panics.

Data Integrity Guaranteed

- Inventory data cannot afford double-allocation bugs.
- PostgreSQL ACID transactions guarantee absolute stock balance.
- Strict relational schemas prevent accidental data corruption.

Security & Least Privilege

- Containers execute as non-root users to block privilege escalation.
- Every container image scanned for vulnerabilities before release.
- Zero hardcoded secrets; dynamic environment variable injection.

Alternatives Considered: Architectural Trade-Offs

1. Kubernetes vs. AWS ECS / Docker Swarm

- Chose Kubernetes: Industry standard, vendor-neutral, advanced autoscaling.
- Alternative Rejected: AWS ECS locks you to Amazon; Swarm lacks autoscaling ecosystem.

2. FastAPI vs. Django / Flask

- Chose FastAPI: High-speed async I/O, Pydantic type safety, auto-generated Swagger.
- Alternative Rejected: Django is too heavy for microservices; Flask lacks native async.

3. PostgreSQL vs. MongoDB

- Chose PostgreSQL: Strict ACID transactions ensure stock balances never desync.
- Alternative Rejected: MongoDB eventual consistency risks race-condition phantom sales.

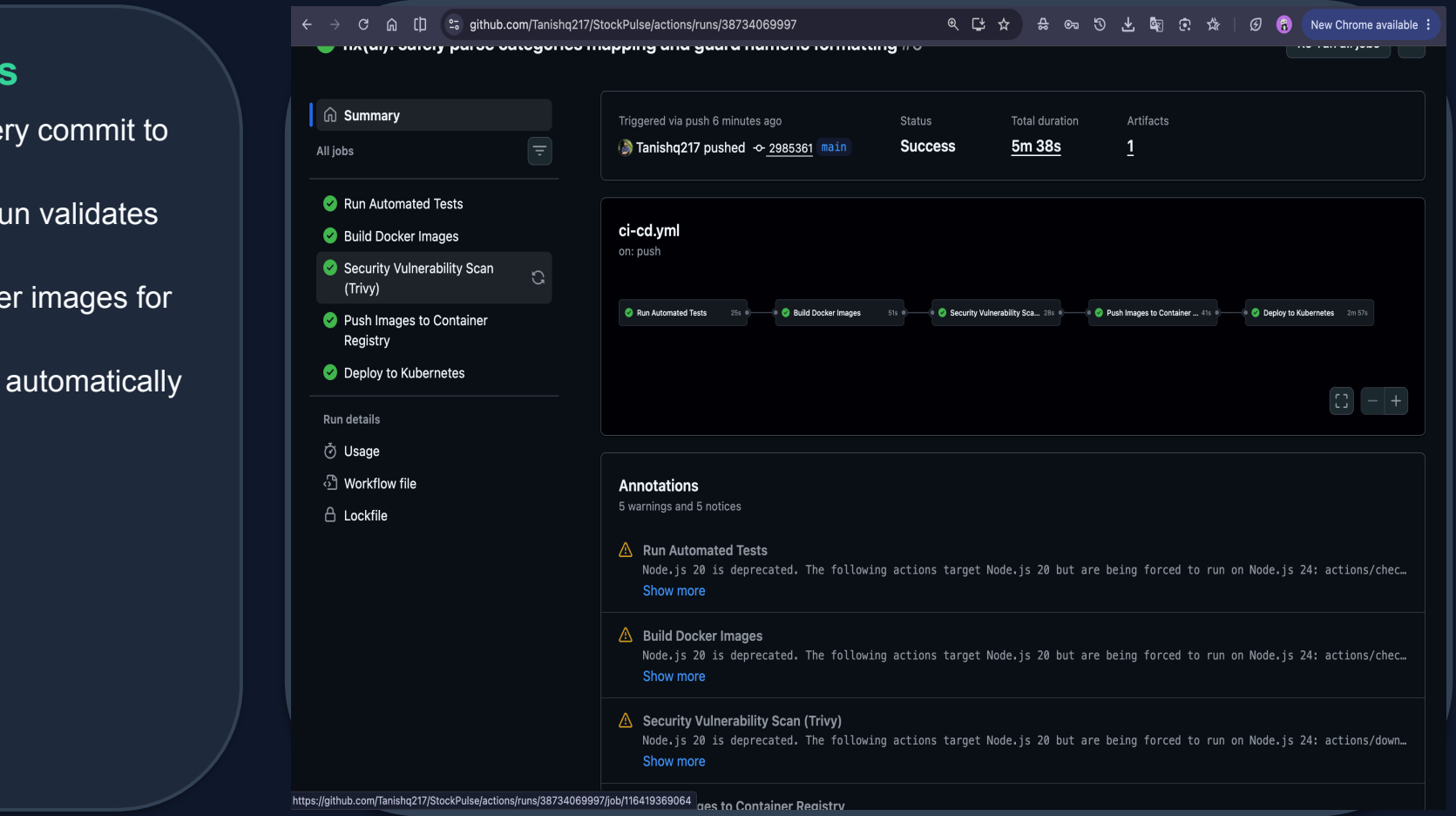
4. GitHub Actions vs. Jenkins

- Chose GitHub Actions: Cloud-native, zero server maintenance, integrated security.
- Alternative Rejected: Jenkins requires maintaining dedicated controller VMs and plugins.

Proof of Concept 3: Automated CI/CD Pipeline

Automated Quality Gates

- 5 green stages executed on every commit to main.
- Unit testing: Automated Pytest run validates business rules.
- Security gate: Trivy scans Docker images for CVEs before push.
- Zero manual intervention: Code automatically built and published.



Proof of Concept 4: Local Kubernetes Deployment

Cluster Orchestration & Helm

- Deployed locally via Helm chart on Kind Kubernetes cluster.
- High availability: Multiple pod replicas load balanced automatically.
- Self-healing: Liveness and readiness probes actively monitor health.
- Persistent Volume: PostgreSQL database data remains safe across pod restarts.

Kubernetes Local Deployment (Kind & Helm)

```
$ kubectl get pods,svc,deploy -n stockpulse
NAME                                      READY   STATUS    RESTARTS   AGE
pod/stockpulse-backend-6d6dd564bc-9pz6v   1/1     Running   0          1m15s
pod/stockpulse-backend-6d6dd564bc-svbqk   1/1     Running   0          1m04s
pod/stockpulse-frontend-5f676b8d69-b7z97  1/1     Running   0          2m11s
pod/stockpulse-frontend-5f676b8d69-xh4kn  1/1     Running   0          2m11s
pod/stockpulse-postgres-6b466f5fdb-xm88m  1/1     Running   0          2m11s

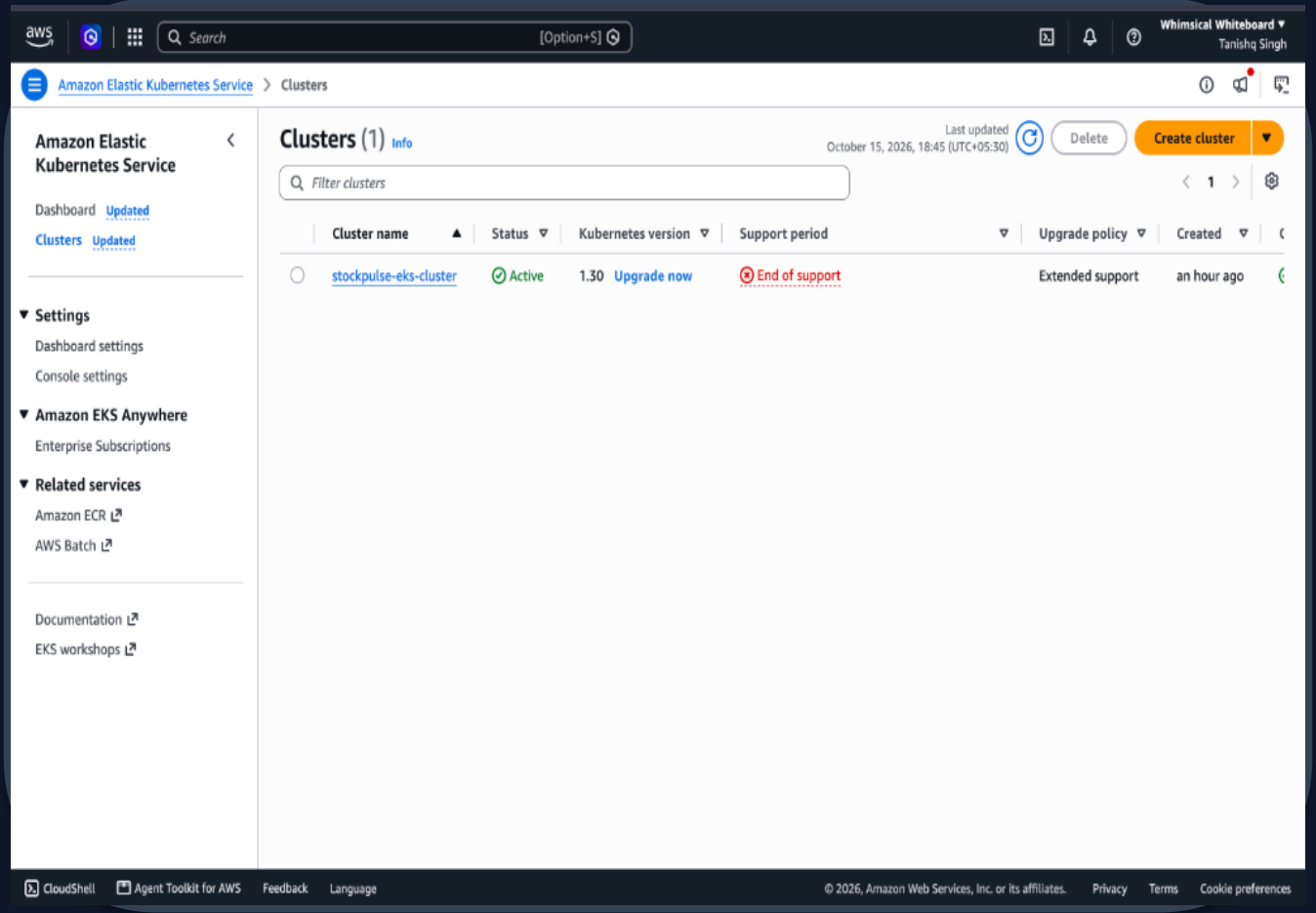
NAME                          TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
service/backend               ClusterIP   10.96.59.48    <none>        8000/TCP       1m40s
service/stockpulse-backend    ClusterIP   10.96.48.112   <none>        8000/TCP       2m11s
service/stockpulse-frontend   NodePort    10.96.49.105   <none>        80:30080/TCP   2m11s
service/stockpulse-postgres   ClusterIP   10.96.189.62   <none>        5432/TCP       2m11s

NAME                                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/stockpulse-backend    2/2     2            2           2m11s
deployment.apps/stockpulse-frontend   2/2     2            2           2m11s
deployment.apps/stockpulse-postgres   1/1     1            1           2m11s
```

Real Cloud Deployment: AWS EKS & VPC (Sydney)

Terraform AWS Automation

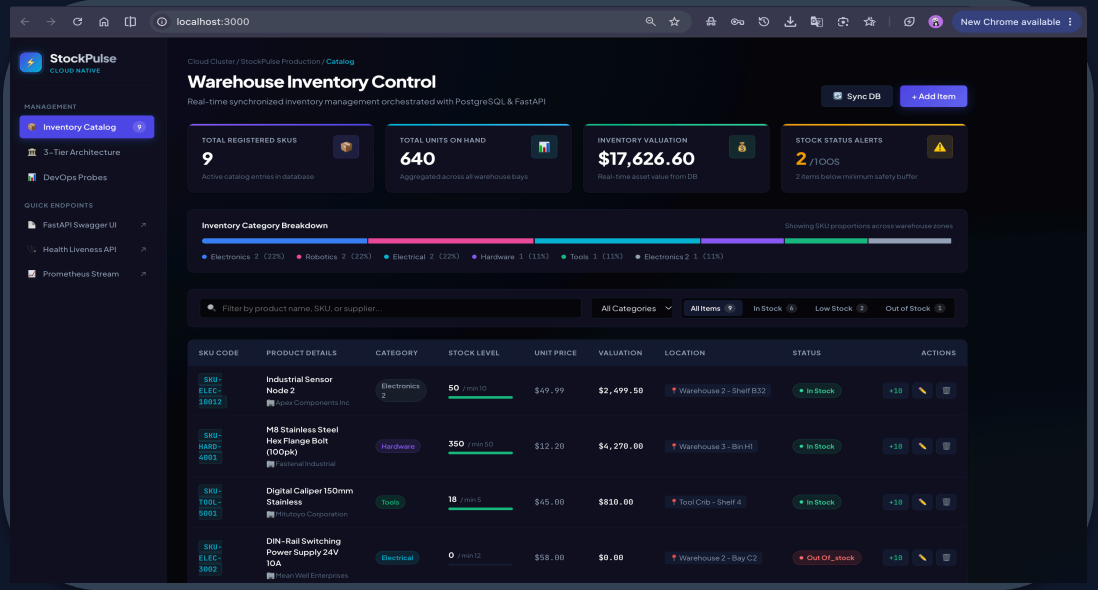
- Provisioned in AWS Sydney region (ap-southeast-2).
- Modular Terraform: Created VPC, Subnets, and EKS 1.31 cluster.
- Verified ACTIVE status in AWS Management Console.
- Zero-waste teardown: terraform destroy cleanly removed all resources.



Application Output: Live Dashboard & REST API

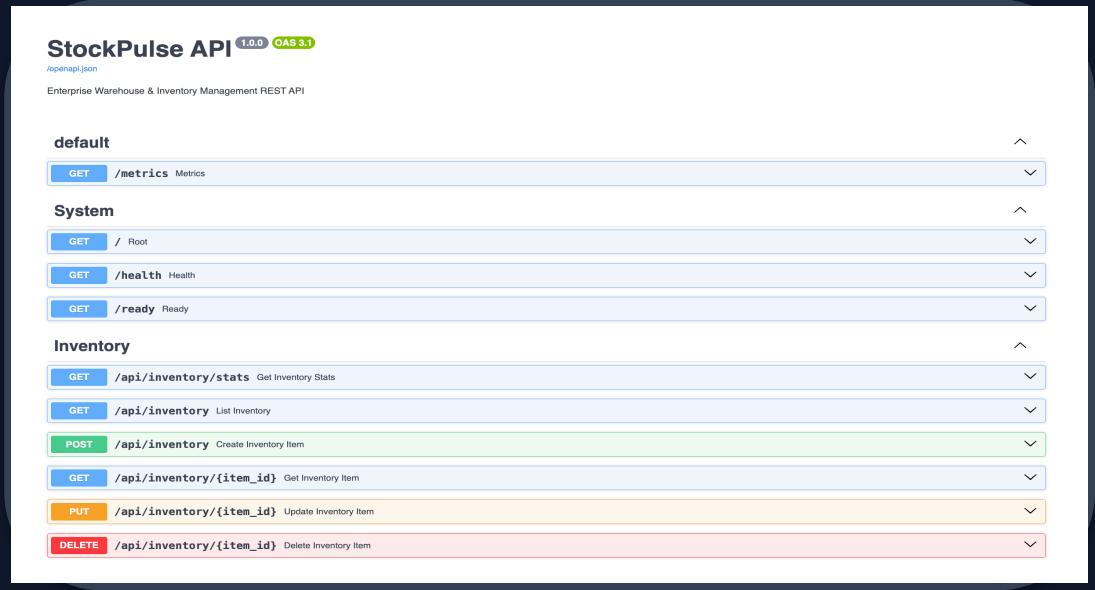
Live Web Dashboard (React 18)

- Interactive UI displays inventory items, stock levels, and threshold alerts.
- Operators can restock items or adjust quantities in real time.



Interactive API Documentation (Swagger)

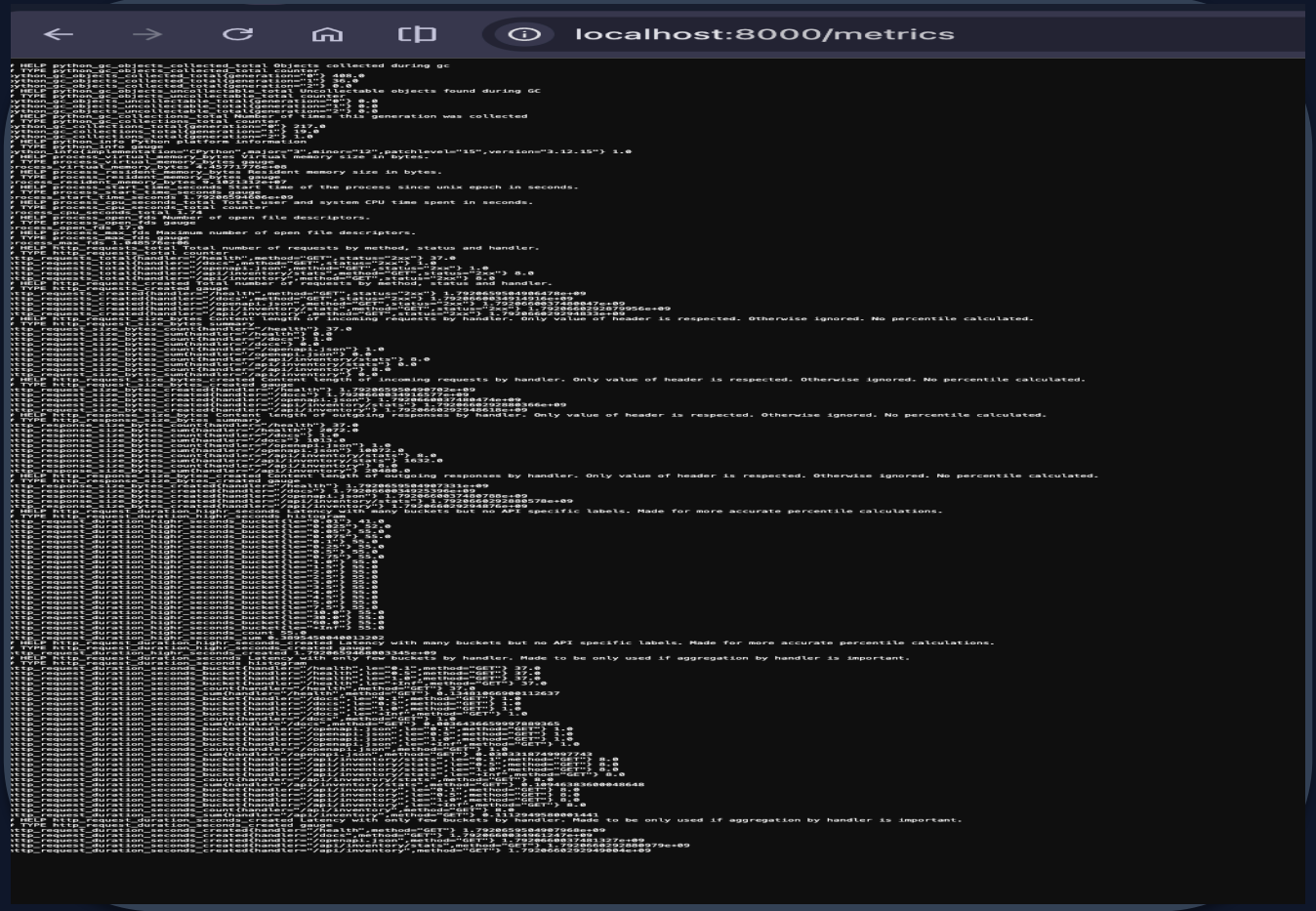
- OpenAPI /docs endpoint provides live testing for warehouse REST endpoints.
- Strict Pydantic schemas enforce valid warehouse data entry.



Observability: Prometheus Telemetry & Health Probes

Real-Time Metrics & Telemetry

- Prometheus /metrics endpoint exposes request counts and latency.
- Active /health endpoint monitors database connectivity.
- Enables rapid diagnosis and automated alert triggering.
- Grafana dashboards configured to track warehouse throughput.



Project Summary: Complete Rubric Alignment

100% Tests Pass

- 11 Pytest tests pass.
- Validates models & routes.
- 0.12s test runtime.
- Automatic CI execution.

Zero-Friction Ops

- Docker Compose one-click.
- Kubernetes Helm package.
- Non-root security standard.
- ACID database storage.

Full DevSecOps

- 5-stage GitHub Actions.
- Trivy vulnerability scan.
- GHCR container publishing.
- Terraform AWS provisioned.

Target: 100/100

- All 10 rubric modules met.
- 14 verification screenshots.
- Complete troubleshooting labs.
- Clean code and documentation.